

《离散数学(2)》图论算法通俗直观解读

叶学翰 整理

天下苦离散教材上的算法描述不说人话久矣！不知道有谁能看懂，反正我是看不懂。所以我手搓了这么一个文档，整理了一下2026崔勇老师图论课上讲过的一些算法，尽量用人能看得懂的语言去描述。全程未使用AI进行文档编辑，仅参考书本和课件。以及，仅个人理解，一派胡言。如有不严谨，还望批评指正。

Warshall 算法

目的：

连通性、道路存在性、道路矩阵 $P = A \vee A^2 \vee \cdots \vee A^n$ 、传递闭包

例：

比方说四个人有认识关系， $A \rightarrow B, B \rightarrow C, C \rightarrow D$
邻接矩阵 A ：

	A	B	C	D
A	0	1	0	0
B	0	0	1	0
C	0	0	0	1
D	0	0	0	0

操作方式：

- 依次选中间人。第一次选 A 。
- 先看 A 这一列，能看出谁认识 A （也就是 A 的直接前驱）；再看 A 这一行，看 A 认识谁（直接后继）。
- 通过这个中间人能建立哪些连接？将新的连接更新上去。这里没变化。
- 然后选 B 作为中间人。看 B 列， A 认识 B 。看 B 行， B 认识 C 。所以 $A \rightarrow C$ 。更新：

	A	B	C	D
A	0	1	1	0
B	0	0	1	0
C	0	0	0	1
D	0	0	0	0

- 然后选 C 当中间人。一样去看它的行和列。更新：

	A	B	C	D
A	0	1	1	1
B	0	0	1	1

C	0	0	0	1
D	0	0	0	0

6. 最后是 D 来当中间人。更新后没变化了。
7. 那就over了！最后更新出来的就是道路矩阵 P 了。

Hierholzer 算法

目的：

构造欧拉回路

操作方式

1. 特别简单。直接去图里找回路就完事。啥回路都行。
2. 还有没用过的边，那就在刚才那个回路上找一个有未用边的点，从它出发再找一个回路。
3. 合并回路。
4. 重复重复，就over了。

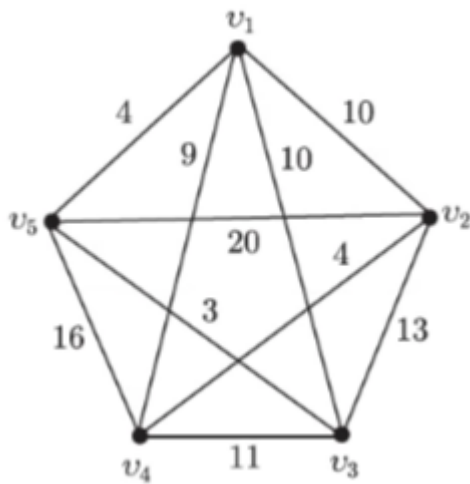
TSP 之 分支定界法

目的：

旅行商问题：给定一个正权完全图，求权值和最小的哈密顿回路。
这里注意，是完全图。如果实际上不是完全图，无所谓，补上的边权值置为 ∞ 就行。

例：

看书本例题



目标：从 v_1 出发，每个点恰好走一次，最后返回 v_1 。找最短哈密顿回路。

操作方式：

1. 先把边权排序。初始界 $d_0 \leftarrow \infty$ 。

$$\begin{array}{cccccccccc} a_{ij} : & a_{53} & a_{42} & a_{15} & a_{14} & a_{12} & a_{13} & a_{34} & a_{23} & a_{45} & a_{25} \\ l_{ij} : & 3 & 4 & 4 & 9 & 10 & 10 & 11 & 13 & 16 & 20 \end{array}$$

2. 下面用DFS方法。按序列依次选边进行深探，直到选够 n 条边。判断是不是构成了哈密顿回路，也就是每个顶点号出现两次，而且只构成一个回路。

3. 如果构成了，那就更新 d_0 。

4. 如果没构成，就继续做深搜、退栈。

5. 一直到所有正确或错误的解都不可能优于 d_0 ，就over。

(个人觉得挺麻烦的反正)

TSP 之“便宜”算法

例：

依旧书本例题

$$\begin{bmatrix} 0 & 18 & 35 & 25 & 27 \\ 18 & 0 & 23 & 21 & 19 \\ 35 & 23 & 0 & 17 & 28 \\ 25 & 21 & 17 & 0 & 24 \\ 27 & 19 & 28 & 24 & 0 \end{bmatrix}$$

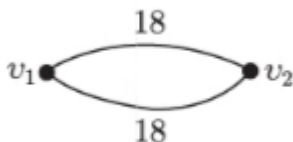
操作方式：

1. 我们的解起初是一个 v_1 的自环。

2. 下一个是谁？看 v_1 和谁最近。这时候得来看矩阵，哪些节点在解里了，就看对应行；如果不在，就看对应列。所以实际上需要看的区域挺少的，下面都会用红色方框标出。

$$\begin{bmatrix} 0 & 18 & 35 & 25 & 27 \\ 18 & 0 & 23 & 21 & 19 \\ 35 & 23 & 0 & 17 & 28 \\ 25 & 21 & 17 & 0 & 24 \\ 27 & 19 & 28 & 24 & 0 \end{bmatrix}$$

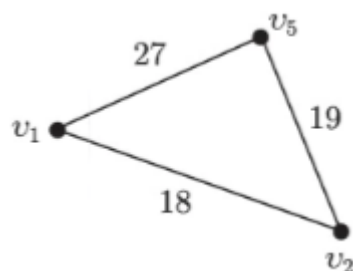
3. 那就选 v_2 呗。至于插在哪里，没啥可说的。那就变成这样：



4. 下一个是谁？看 v_1 、 v_2 和谁最近：

0	18	35	25	27
18	0	23	21	19
35	23	0	17	28
25	21	17	0	24
27	19	28	24	0

5. 那就选 v_5 。插在哪里还是没啥可说。变成下面这样：

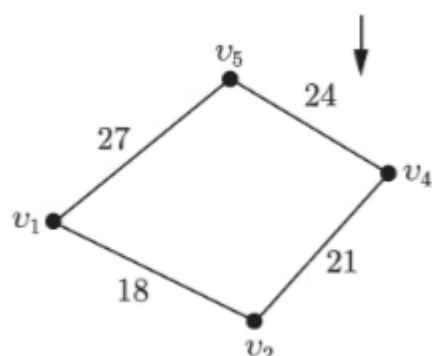


6. 下一个是谁？

0	18	35	25	27
18	0	23	21	19
35	23	0	17	28
25	21	17	0	24
27	19	28	24	0

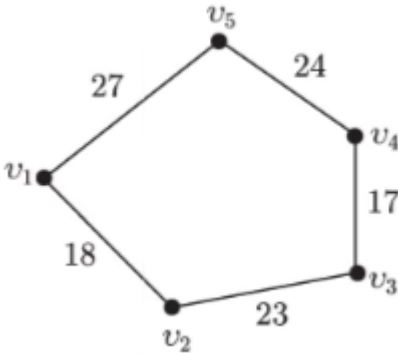
7. 选 v_4 。这时候就要考虑插在哪里了，是 v_1 v_2 之间还是 v_5 v_2 之间？

8. 权衡一下： $21 + 25 - 18 > 21 + 24 - 19$ ，所以插在 v_2 v_5 之间。变成：



9. 继续选下一个，然后一样看插在哪里。这里 $17 + 23 - 21 < 17 + 28 - 24$ ，所以插 $v_2 v_4$ 之间。

0	18	35	25	27
18	0	23	21	19
35	23	0	17	28
25	21	17	0	24
27	19	28	24	0

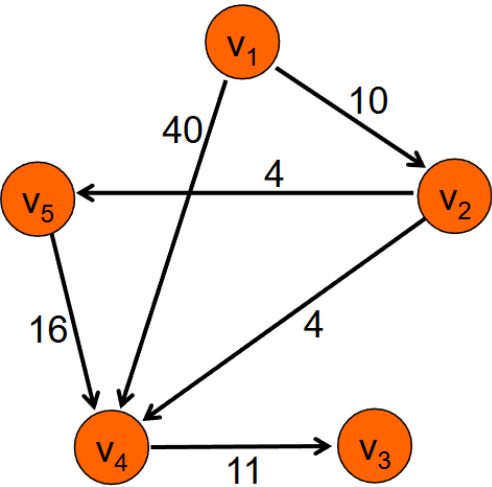


10. 总算是over了！
(个人觉得比分支定界还要麻烦，到底便宜在哪里.....)

Dijkstra 算法

目的：
最短路问题，适用于非负权图。

例：
直接用课件例题



操作步骤：
直接上图，很清晰了（反正比书上的算法好理解一百倍）

$\pi(1)$	$\pi(2)$	$\pi(3)$	$\pi(4)$	$\pi(5)$	-S	访问
0	∞	∞	∞	∞	1,2,3,4,5	1
0	10	∞	40	∞	2,3,4,5	2
0	10	∞	14	14	3,4,5	4
0	10	25	14	14	3, 5	5
0	10	25	14	14	3	3
0	10	25	14	14	Φ	-

注意每次选访问节点的时候，要选 π 最小的。

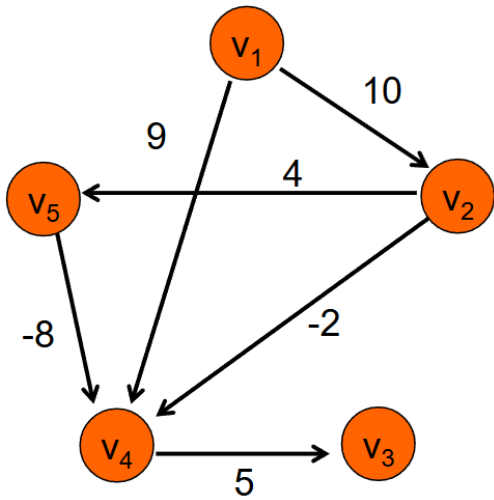
Bellman-Ford 算法

目的：

最短路问题，适用于任意边权图。

例：

依旧课件例题



操作步骤：

还是直接看图。概括来说就是松弛 $n - 1$ 轮，或者松弛到不变为止。

$\pi(1)$	$\pi(2)$	$\pi(3)$	$\pi(4)$	$\pi(5)$	k
0	∞	∞	∞	∞	0
0	10	∞	8	14	1
0	10	13	6	14	2
0	10	11	6	14	3
0	10	11	6	14	4

关键路径算法

目的：

在PERT图中，找到最长路径。

操作步骤：

比较简单，直接写了。

1. 先对节点序号做拓扑排序。

2. 按序号依次算最长路径，全部依赖于其直接前驱。
3. 算到最后一个节点就over了，EEZZ啊。

拓展：最晚完工时间 & 最大允许延误时间

1. 已知关键路径和最早完工时间。
2. 按序号逆序，可计算出**节点最晚完工**时间。
3. 依此，减去对应边权，可计算出**工序最晚启动**时间。
4. 通过关键路径，可知**工序最早启动**时间。
5. 作差，即得最大允许延误时间。到这里就over了，依旧不难。

中国邮路构造方法

目的：

在正权连通图中，求一条从某点出发、经过每条边至少一次、最后回到出发点的闭合路线，使总长度最小。
(中译中就是欧拉回路找不到，退而求其次可以重复走呗。没活硬整我说白了。)

操作方法：

1. 找到所有奇度顶点，数量除以2就是要添加的重边数量。
2. 去添加重边，尽可能让重边边权和小，就over了。

支撑树计数算法：Matrix-Tree 定理

目的：

计算连通图 G 中的支撑树个数。
(感觉其他东西什么定理的都不用管，会算就行了)

一、有向连通图的树计数

$$\tau(G) = \det(B_k B_k^T)$$

- 不含边 e 的支撑树数：删除 e 后计数，即 $\tau(G - e)$ 。
- 必含边 e 的支撑树数：收缩 e 后计数，即 $\tau(G/e)$ 。

二、无向连通图的树计数

给每条边随便加个方向就行了。

三、有向连通图根树的计数

以 v_k 为根的根树的数目是 $\det(\vec{B}_k B_k^T)$
其中 $\vec{B}_k$ 表示将 B_k 中全部1改成0

- 以 v_0 为根，必含边 e 的根树数目：
计算含 e 的根树数，减去不含 e 的根树数。
或，若 $e = (u, v)$ ，任何其他 (t, v) 形式的边都不会出现，去除后计数即可。

基本回路矩阵构造方法

1. 找一棵支撑树。
2. 对每一条余树边，加入支撑树中，必会形成一个回路，回路方向与这条余树边相同。
3. 共 $m - n + 1$ 次，做完就over了。

基本割集矩阵构造方法

1. 找一棵支撑树。
2. 对每一条树枝边，可以和某些余树边一起构成一个割集，割集方向与这条树枝边相同。
3. 共 $n - 1$ 次，做完就over了。

Huffman 算法

过程过于简单，没啥可说的。

就注意一下， k 进制编码情形下，叶子节点数量 n 需要满足 $n \equiv 1(\text{mod } k)$ 。

Kruskal 算法

目的：

在赋权连通图中找到最短树。

操作步骤：

1. 将边权值从小到大排序。
2. 从空边集开始。
3. 依次考察边，若加入后不形成回路，则加入。
4. 直到选出 $n - 1$ 条边。

Prim 算法

目的：

同样是在赋权连通图中找到最短树。

操作步骤：

1. 从任一顶点开始，令已选顶点集为 U 。
2. 每次选择一条连接 U 和 $V - U$ 的最小权边。
3. 将该边和新顶点加入树。
4. 重复直到覆盖所有顶点。